Name: _____ Register No: _____

Experiment 6

Question: Time Series Analysis of Website Traffic.

Code

```
import json
import re
from collections import Counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

def exp_6():
    rng = np.random.default_rng(3)
    dates = pd.date_range("2024-01-01", "2025-12-31", freq="D")
    trend = np.linspace(1000, 1800, len(dates))
    weekly = 150 * np.sin(2 * np.pi * dates.dayofweek / 7)
    noise = rng.normal(0, 60, len(dates))
    visits = (trend + weekly + noise).round()
    ts = pd.Series(visits, index=dates, name="Visits")

    print(ts.head(), "\n")
    print(ts.describe(), "\n")

    # Smoothing
    rolling7 = ts.rolling(7).mean()
    rolling30 = ts.rolling(30).mean()

    # Resampling
    monthly = ts.resample("ME").sum()      # use "M" on older pandas
    print("Monthly traffic:\n", monthly.head(), "\n")

    # Day of week pattern
    by_day = ts.groupby(ts.index.day_name()).mean().round(1)
    print("Average visits by weekday:\n", by_day, "\n")

    # Plots
    fig, ax = plt.subplots(3, 1, figsize=(11, 10))
    ts.plot(ax=ax[0], alpha=0.4, label="Daily")
    rolling7.plot(ax=ax[0], label="7-day avg")
    rolling30.plot(ax=ax[0], label="30-day avg")
    ax[0].set_title("Website Traffic with Moving Averages")
    ax[0].legend()
    monthly.plot(kind="bar", ax=ax[1], title="Monthly Visits")
    ax[1].set_xticklabels([d.strftime("%b %y") for d in monthly.index], rotation=45)
    by_day.reindex(["Monday", "Tuesday", "Wednesday", "Thursday",
                    "Friday", "Saturday", "Sunday"]).plot(
        kind="bar", ax=ax[2], title="Average Visits by Weekday")
    plt.tight_layout()
    plt.show()

    # Decomposition into trend + seasonality + residual
    try:
        from statsmodels.tsa.seasonal import seasonal_decompose
        seasonal_decompose(ts, model="additive", period=7).plot()
        plt.show()
    except ImportError:
        print("Install statsmodels for decomposition: pip install statsmodels")

    # Stationarity check (ADF test)
    try:
        from statsmodels.tsa.stattools import adfuller
        stat, p, *_ = adfuller(ts)
        print(f"ADF statistic = {stat:.3f}, p-value = {p:.3f} ->",
              "stationary" if p < 0.05 else "non-stationary (has trend)")
    except ImportError:
        pass


exp_6()
```

Output

```
2024-01-01    1122.0
2024-01-02     965.0
2024-01-03    1174.0
2024-01-04    1034.0
2024-01-05     912.0
Freq: D, Name: Visits, dtype: float64

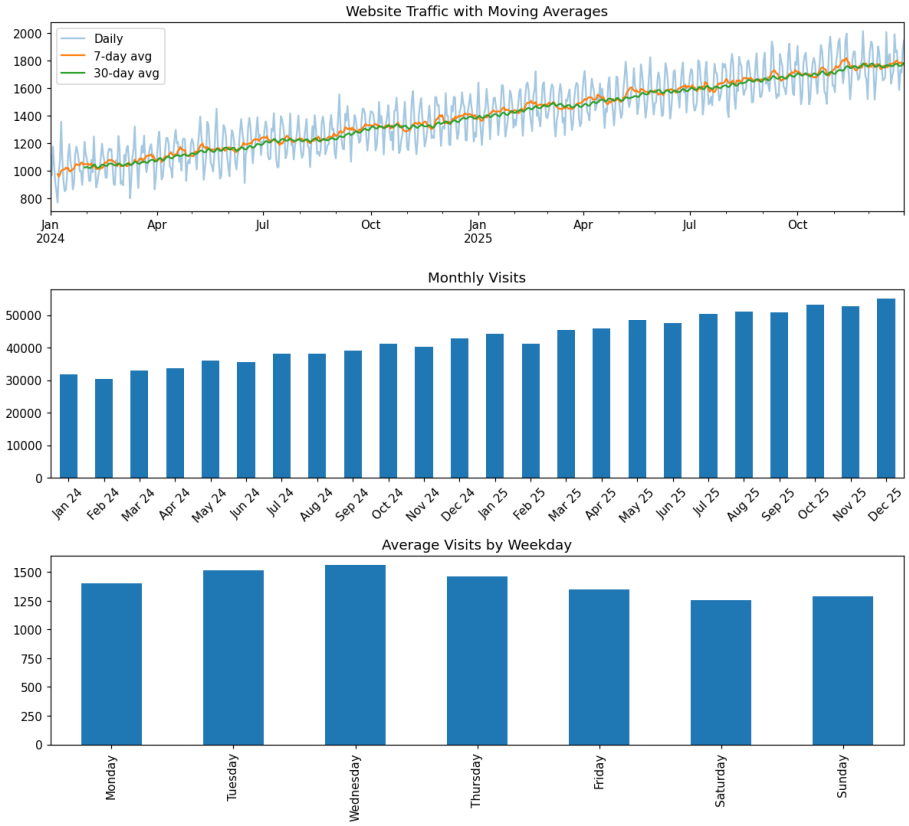
count     731.000000
mean     1403.834473
std       262.258873
min       768.000000
25%      1196.500000
50%      1404.000000
75%      1602.500000
max      2014.000000
Name: Visits, dtype: float64

Monthly traffic:
 2024-01-31    31823.0
2024-02-29    30396.0
2024-03-31    32916.0
2024-04-30    33645.0
2024-05-31    36138.0
Freq: ME, Name: Visits, dtype: float64

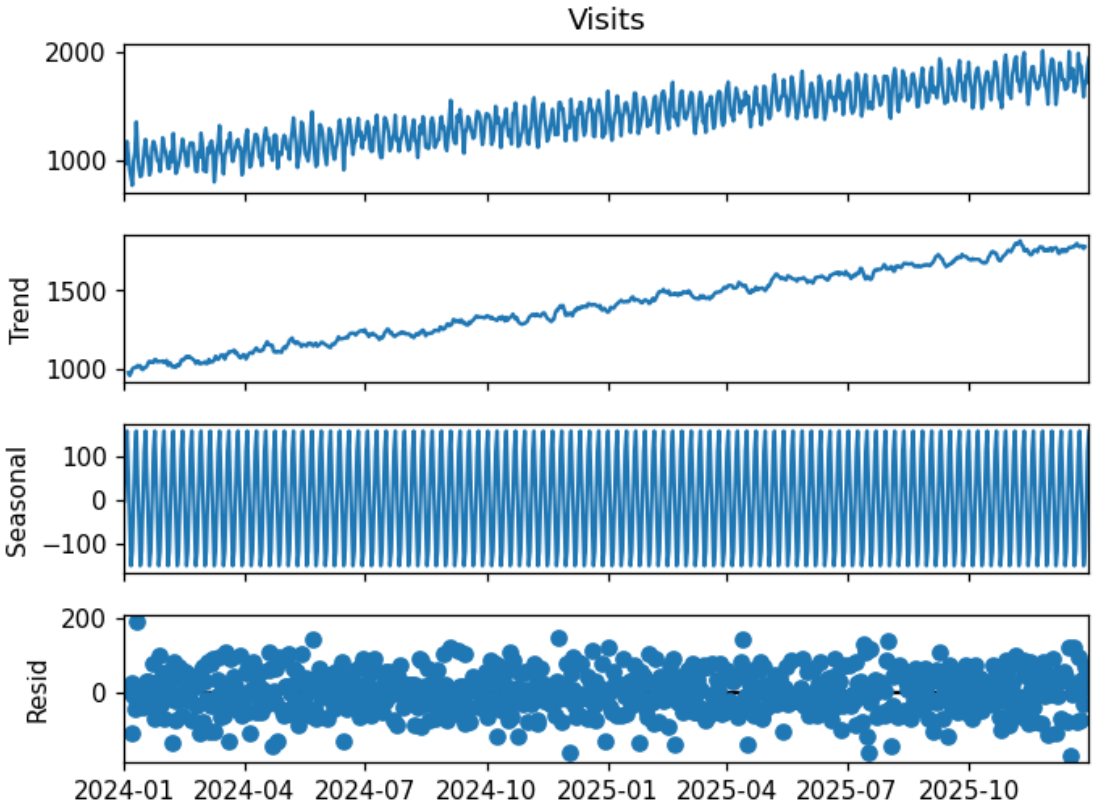
Average visits by weekday:
 Friday       1348.3
Monday       1400.8
Saturday     1254.0
Sunday       1288.4
Thursday     1458.7
Tuesday      1512.8
Wednesday    1561.4
Name: Visits, dtype: float64

ADF statistic = -0.612, p-value = 0.868 -> non-stationary (has trend)
```

Graph



Graph


Visits
2000
1000
Trend
1500
1000
Seasonal
100
0
−100
Resid
200
0
2024-01
2024-04
2024-07
2024-10
2025-01
2025-04
2025-07
2025-10